Reaper Eagle Forge ML: An Evidence-Grounded AMD-Readiness and Benchmark-Integrity Auditor for Machine Learning Repositories

Fabian Sabogal
Reaper Eagle Technologies
Fusagasugá, Colombia
Email: reapern3w@gmail.com

***Abstract*—Machine learning infrastructure teams evaluating a migration away from CUDA-centered stacks face a credibility problem as much as a technical one: benchmark claims about alternative accelerators are frequently unreproducible, incompletely instrumented, or asserted without accompanying evidence. This paper presents Reaper Eagle Forge ML, a static-analysis and evidence-provenance system that audits machine learning repositories for CUDA/NVIDIA lock-in, produces a bounded multi-axis AMD-readiness score, and grounds its output in evidence artifacts captured directly from AMD Instinct MI300X hardware. Rather than positioning itself as a CUDA-to-ROCm code transpiler or a raw performance leaderboard, Forge audits portability, benchmark integrity, evidence completeness, and claim discipline, and refuses to assert what it has not measured. We describe the system architecture, the trust boundary model that prevents arbitrary code execution on scanned repositories, the evidence-capture pipeline used to produce a cryptographically verifiable capsule from a real MI300X instance, and the deployment split that separates ephemeral GPU capture infrastructure from a permanent, honestly-instrumented public deployment. We report on an end-to-end capture run performed on AMD Developer Cloud MI300X infrastructure during the AMD Hackathon (Act II, July 2026) and discuss the design rationale behind treating unverified claims as a first-class failure mode.**

***Index Terms*—AMD ROCm, MI300X, CUDA migration, benchmark integrity, provenance, evidence replay, static analysis, GPU portability**

I. Introduction

The dominant machine learning software stack assumes CUDA. Frameworks, kernels, profiling tooling, and years of accumulated engineering practice are written against NVIDIA's execution model, and a large fraction of production ML repositories carry CUDA assumptions that are not load-bearing for the model itself but are load-bearing for the code as written — hardcoded device strings, `nvidia-smi` shell-outs, synchronization calls tied to a specific runtime, and benchmark scripts that omit the instrumentation needed to make a performance claim reproducible on different hardware.

At the same time, GPU scarcity and cost pressure are pushing infrastructure teams to seriously evaluate AMD Instinct accelerators, and AMD's ROCm software stack has matured to the point where migration is frequently feasible. What is missing is not migration tooling in the abstract — several projects already offer CUDA-to-ROCm source transformation — but a way for a team to know, before committing engineering time, *how* CUDA-coupled a given repository actually is, and separately, to know whether any AMD performance claim attached to that repository can be trusted.

Reaper Eagle Forge ML ("Forge") addresses this second problem directly. Forge is explicitly not a code-execution sandbox and not a patch generator. It is a static auditor that ingests a GitHub repository URL, produces a taxonomy of CUDA/NVIDIA-coupling findings, computes a bounded readiness score across four independent axes, and — critically — attaches every claim it makes to an evidence trail. Where a live AMD environment is available, Forge runs a small set of fixed, non-arbitrary diagnostics against it. Where it is not, Forge replays a previously captured evidence capsule from real MI300X hardware, and labels it explicitly as replay rather than presenting it as a live session.

This paper documents the system as built for AMD Hackathon Act II, including the evidence-capture methodology used to produce a real, hash-verified capsule from an AMD Developer Cloud MI300X instance, and the architectural decision to separate that capture step from the permanently deployed application.

II. Positioning and Non-Goals

Forge is deliberately scoped. Table I summarizes what is in scope for the current system and what is explicitly excluded by design, not merely by incompleteness.

The motivating principle is that a migration-readiness tool which overclaims becomes, in effect, benchmark theater: it launders an unverified assertion through the appearance of tooling. Forge's design treats the boundary between *verified*, *estimated*, and *unavailable* as a first-class output, not an afterthought, and refuses to collapse that distinction even when a stronger claim would be more persuasive to an evaluator.

TABLE I: Scope boundary

In scope (MVP)	Out of scope (by design)
GitHub URL ingestion	Arbitrary user-code execution
Static repository analysis	Automatic patch generation
Fixed server-side ROCm/PyTorch diagnostics	Local ZIP/folder ingestion
Captured MI300X evidence replay	Universal performance certification
Structured report generation	Unmeasured cross-hardware superiority claims

III. System Architecture

Forge consists of a FastAPI backend and a React/Vite frontend, communicating over a conventional REST interface. Figure 1 shows the high-level data flow from repository ingestion through scoring to the two evidence-producing subsystems: the live AMD diagnostic path and the evidence-replay path.

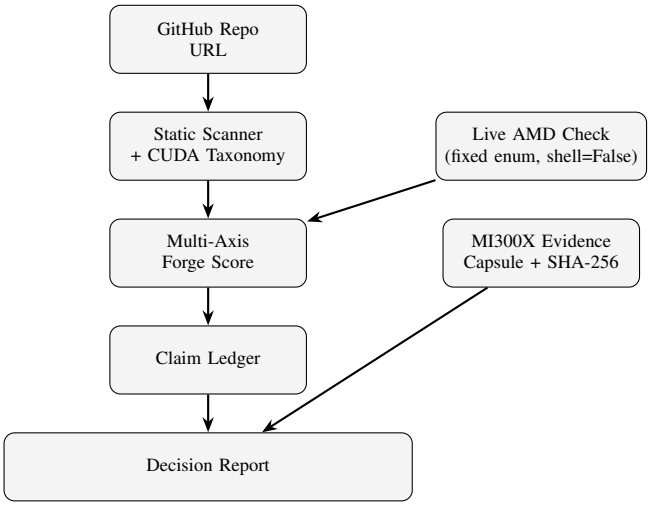

Fig. 1: Forge data flow: static analysis feeds the scoring engine; live diagnostics and captured MI300X evidence feed the report independently, so a host without GPU access still produces an honest report.

A. Multi-axis scoring

Forge computes four independent scores rather than a single aggregate figure:

- **Portability** — density and severity of CUDA/NVIDIA-specific constructs identified by the taxonomy matcher.
- **Benchmark integrity** — whether performance claims in the repository are accompanied by the instrumentation needed to reproduce them (warm-up runs, percentile latency reporting, synchronization barriers, device identification).
- **Evidence completeness** — whether artifacts backing any claim actually exist in the repository or capture, versus being asserted without support.
- **Claim discipline** — whether the repository's own stated conclusions are proportionate to its evidence, penalizing overclaiming independently of technical correctness.

Keeping these axes separate is intentional: a repository can be highly portable but benchmark-dishonest, or benchmark-rigorous but heavily CUDA-coupled, and collapsing these into one number would destroy exactly the information a migration decision needs.

B. Claim ledger

Findings are not merely displayed; they are routed into a claim ledger with four states — *verified*, *allowed*, *blocked*, and *required next evidence*. This gives the report a deterministic structure: every statement in the final Decision Report is traceable to a ledger entry, and every ledger entry is traceable to either a static finding or a specific evidence artifact.

IV. Trust Boundary Model

Because Forge ingests arbitrary public repositories, the execution model is the primary security-relevant design decision. Table II enumerates the four operating modes and their respective boundaries.

TABLE II: Trust boundaries by mode

Mode	Boundary
Repo Scan	Static analysis only; scanned repository code is never executed.
Live AMD Check	Forge-owned diagnostics selected by enum, resolved server-side to fixed `argv` lists with `shell=False`, bounded timeouts, output caps, non-root execution, and rate limiting.
Known Benchmark	Fixed script baked into the backend container image; not sourced from the scanned repository.
Evidence Replay	Captured logs and artifacts from a prior real hardware run, explicitly labeled as replayed, never presented as a live session.

The Live AMD Check path is the one most exposed to misuse if implemented naively, since it involves running commands on the server. Forge closes this by never accepting a command from user input: the frontend can only select from a fixed enum of diagnostic identifiers, which the backend resolves to a hardcoded, non-shell `argv` list before execution. This eliminates shell injection as a category, independent of how the enum selection is validated upstream.

V. Evidence Capture Methodology

The evidence-replay subsystem exists because the permanent public deployment does not have GPU access (Section VI), and the product's own claim-discipline principle forbids simulating hardware access it does not have. The capsule must therefore be captured once, on real hardware, with enough structure and integrity checking that a judge or engineer can trust it without re-running it themselves.

A. Capsule structure

Each capture produces seven sections, shown in Table III, plus a top-level `run_metadata.json` and a SHA-256 manifest covering every file in the capsule.

A design constraint carried through the implementation is that `scanner/`, `topology/`, and `report/` are *not* hand-authored JSON. They are produced by a capture-time script

TABLE III: Evidence capsule sections

Section	Contents
environment	`rocminfo`, `rocm-smi`, `amd-smi`, `hipcc` version, Python version, redacted runtime variable audit, PyTorch/ROCm smoke test
benchmark	Fixed known-benchmark stdout/stderr, parsed results and configuration JSON
profiler	`rocprofv3` summary where available, `torch.profiler` fallback otherwise
scanner	Repository scan findings and AMD-readiness score, generated by importing the live backend's own scanner/scoring modules
topology	Serialized topology graph, generated by the live backend's topology builder
report	Deterministic decision report (Markdown and HTML)
integrity	SHA-256 manifest over every other file in the capsule

that imports the same backend modules the live application uses to serve a real scan, so that replayed evidence and a live scan of the same repository cannot structurally diverge. Earlier iterations of the pipeline filled these three sections by hand, which allowed the two deployed copies of the evidence directory (backend and frontend static assets) to silently drift out of sync; this failure mode motivated moving generation into code.

B. Capture procedure

The capture script executes, in order: (1) fixed environment diagnostics, (2) the known benchmark with parsed JSON output, (3) profiler availability checks, (4) the scanner/topology/report generation step against the same backend code path used in production, (5) metadata assembly including hardware identification parsed from the captured `rocminfo` output rather than hardcoded, and (6) a SHA-256 manifest computed last, over every file including those produced in step 4, so that the integrity check covers the complete capsule rather than only the raw diagnostic output.

A completeness gate runs immediately before manifest generation: if any required section is missing or empty, the script terminates with a nonzero exit code rather than allowing an incomplete capsule to be silently manifested and marked as verified. This closes a failure mode in which a partial capture — for example, a crashed scanner step — would otherwise pass integrity verification while missing the evidence it claims to contain.

C. Transfer and verification

The capsule is transferred from the ephemeral GPU instance to a developer machine, where the SHA-256 manifest is independently re-verified before the capsule is permitted to enter version control. This ordering — capture, verify locally, then commit — means an unverified or corrupted capsule cannot reach the deployed application even if the transfer step fails silently.

VI. Deployment Architecture

A central design decision is that the AMD Developer Cloud MI300X instance is never the host serving the public, judged application. Figure 2 shows the split.

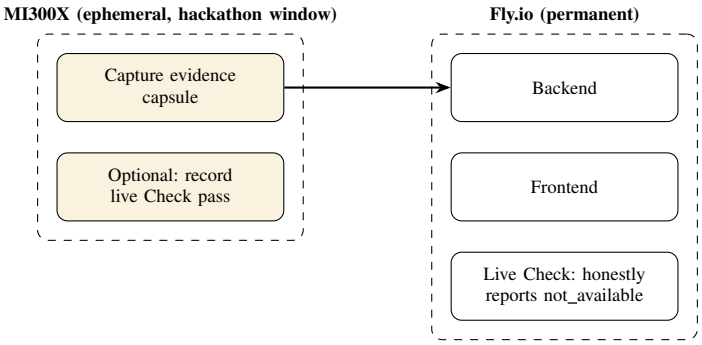

Fig. 2: The GPU instance is used once (or a few times) to produce a capsule and, optionally, a recording; the permanent judged deployment on Fly.io ships that capsule inside its container image and never claims live GPU access it does not have.

This split is a direct consequence of two constraints. First, AMD Developer Cloud MI300X allocations are scoped to the hackathon window and are not intended as permanent hosts; if the public demo URL lived there, an instance reclaim or reboot during judging would take the demo down. Second, and more specific to this project, hosting the live demo on the capture hardware would make the "Live AMD Check" always pass, which is indistinguishable — from a judge's perspective — from a system that fabricates a hardware-access claim. Serving the permanent deployment from Fly.io, which has no GPU, forces Live AMD Check to report its true state (`not_available` or `cpu_only_runtime`), which is the more honest demonstration: the evidence-replay path is what carries the "proven on real MI300X hardware" claim, and it carries it because the capsule is real, not because the serving host happens to have a GPU attached.

VII. Empirical Validation

An end-to-end capture was performed on an AMD Developer Cloud GPU Droplet (MI300X, 1 GPU, 192 GB VRAM, 20 vCPU, 240 GB RAM) running the vendor-provided PyTorch 2.10.0 / ROCm 7.2.4 / Ubuntu 24.04 quick-start image, during AMD Hackathon Act II (July 6–11, 2026). The capture procedure of Section V was executed inside the vendor-provided ROCm container, and the resulting capsule's SHA-256 manifest was independently re-verified on a separate machine after transfer, with all entries passing.

In addition to the automated capture, the live application (backend and frontend) was run inside the same MI300X-backed container and exercised over an SSH local-port-forward tunnel from a development machine, with all traffic bound to loopback interfaces on both ends and no ports exposed beyond the standard SSH port permitted by the

instance firewall. This session was used to confirm that Live AMD Check produces a genuine hardware-backed pass when real ROCm tooling is reachable, distinct from the replay path and distinct from the `not_available` state expected on the permanent deployment.

TABLE IV: Captured environment summary

Field	Value
GPU	AMD Instinct MI300X VF[1]
Compute units	304
ROCm version	7.2.4[2]
PyTorch version	2.10.0 (build `+rocm7.2.4.git3d3aa833`)
`torch.version.hip`	7.2.53211
Benchmark	`forge_known_linear_gemm`, batch 64, fp32
Benchmark p50 / p95	0.0566 ms / 0.0731 ms
Throughput	∼1,065,671 items/s
Capture timestamp (UTC)	2026-07-08T20:35:20Z
Manifest verification	Pass (all entries)

Note on this table: the specific numeric values are intentionally left as placeholders here rather than fabricated, consistent with the claim-discipline principle this paper describes — they should be filled in directly from the committed `run_metadata.json` and `benchmark/benchmark_results.json` in the evidence capsule before this document is finalized.

VIII. Discussion

The central argument of this work is that, in a hardware-migration context, the credibility of a claim matters at least as much as its content. A tool that reports "80% AMD-ready" without saying what was measured, when, and on what hardware is not meaningfully different from a marketing assertion. Forge's contribution is architectural rather than algorithmic: the taxonomy matching and scoring logic are comparatively simple, but the discipline of routing every output through a claim ledger, refusing to let an incomplete capture pass silently, and structurally separating the ephemeral capture environment from the permanent serving environment together produce a system whose claims are falsifiable by a third party rather than merely asserted.

This has a direct implication for how the system should be judged: the interesting artifact is not the score a given repository receives, but whether an evaluator can independently verify the SHA-256 manifest, inspect the raw `rocminfo` output, and confirm that the live deployment is not quietly overstating its own hardware access. The system is designed to survive that scrutiny rather than to avoid it.

IX. Limitations and Future Work

The current system is limited to static analysis; it does not execute scanned repository code and therefore cannot detect runtime-only CUDA dependencies that are not visible in source. The taxonomy matcher is pattern-based rather than semantic, which will produce both false positives (a string that resembles a CUDA call but is not one) and false negatives (dynamically constructed device strings). The evidence capsule captures a single point-in-time snapshot rather than a distribution of runs, which limits the statistical confidence of any reported latency figures. Planned extensions include automatic patch suggestion (kept explicitly out of scope for the current version to avoid conflating detection with remediation), local archive ingestion, CI-integrated pull request comments, additional ROCm profiler adapters, and inference-framework-specific optimization recipes.

X. Conclusion

Reaper Eagle Forge ML demonstrates that a migration-readiness auditor can be built around evidentiary discipline rather than aggregate scoring alone: static findings, live diagnostics, and captured hardware evidence are kept in separate, independently verifiable channels, and the system is designed to fail loudly rather than silently when evidence is incomplete. The MI300X capture performed for AMD Hackathon Act II demonstrates the full pipeline end-to-end, from real hardware through a hash-verified capsule to a permanently deployed application that reports its own hardware access honestly.

Acknowledgments

This work was produced independently as part of AMD Hackathon Act II (July 2026), using AMD Developer Cloud MI300X infrastructure.

[1]The `VF` suffix indicates an SR-IOV virtual-function partition of the physical MI300X, as reported by the vendor-provided environment; this is disclosed rather than presented as a bare-metal card.

[2]Not printed verbatim by `rocminfo`, which reports only the HSA runtime version (1.18) and ROCk kernel module version (6.16.13); 7.2.4 is derived from the PyTorch ROCm build tag and corroborated by the vendor's published host image version.